```
def ai_filter_candidates(candidates_list, requirements):
    """
    AI Filtering Logic:
    1. Calculate scores based on Skills match, CGPA, and Experience.
    2. Rural candidates get a priority weight.
    3. Social Category candidates (SC, ST, OBC, MBC) get a priority weight.
    4. Return ALL candidates sorted by score (not just top 5)
    """
    scored = []

    for cand in candidates_list:
        score = 0

        # Handle potential None values for strings
        skills_raw = cand.get('skills') or ''
        social_cat_raw = cand.get('social_category') or ''
        rural_raw = cand.get('rural') or ''
        rural_urban_raw = cand.get('rural_urban') or ''
        experience_raw = cand.get('experience') or ''

        # Skill Match Score (assuming comma separated)
        cand_skills = set(s.strip().lower() for s in skills_raw.split(','))
        req_skills = set(s.strip().lower() for s in requirements.get('skills', '').split(','))
        match_count = len(cand_skills.intersection(req_skills))
        score += match_count * 15

        # CGPA Score (Max 10)
        cgpa = cand.get('cgpa') or 0.0
        score += (float(cgpa) * 5) # Up to 50 points

        # Experience weight
        if len(experience_raw) > 10:
            score += 10

        # Rural Priority Weight
        is_rural = rural_raw == 'Yes' or rural_urban_raw == 'Rural'
        if is_rural:
            score += 20

        # Social Category Priority Weight
        # "SC", "ST", "OBC", "MBC", "BC"
        social_category = social_cat_raw.upper().replace('.', '')
        is_reserved = social_category in ['SC', 'ST', 'OBC', 'MBC', 'BC', 'MBC/DNC']
        if is_reserved:
            score += 20

        scored.append({'data': cand, 'score': score, 'is_rural': is_rural, 'is_reserved': is_reserved})
```

```
    # Sort by score (highest first)
    scored_sorted = sorted(scored, key=lambda x: x['score'], reverse=True)

    # Return ALL candidates sorted by score
    return scored_sorted


def get_top_candidates(all_candidates, limit=5):
    """Get top N candidates from the sorted list"""
    return all_candidates[:limit]
```

```
"""
AI-Driven Automatic Candidate Selection System
Handles automatic filtering, ranking, and status assignment with notifications
"""

import sqlite3
import datetime
import time
from database import get_connection
from ai_engine import ai_filter_candidates
from email_service import send_update_to_candidate, send_hr_announcement
import logging

# Setup logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('ai_auto_selector.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)


def process_company_applications(company_name):
    """
    AI-driven processing for a specific company:
    1. Filter all 'Applied' candidates using AI
    2. Rank them by AI score
    3. Assign statuses: 1st = Selected, 2nd = Shortlisted, 3rd+ = Waiting List
    4. Send notifications in order: 3rd → 2nd → 1st
    5. Notify HR when filtering is complete
    """
    try:
        conn = get_connection()
        cursor = conn.cursor()
```

```
logger.info(f"=" * 80)
logger.info(f"Starting AI processing for company: {company_name}")

# Get all 'Applied' candidates for this company
applied_candidates = cursor.execute("""
    SELECT a.*, u.name, u.email, u.phone, u.district, u.rural, u.social_category, u.dob
    FROM applications a
    JOIN users u ON a.user_id = u.id
    WHERE a.company = ? AND a.status = 'Applied'
    ORDER BY a.created_at ASC
""", (company_name,)).fetchall()

if not applied_candidates or len(applied_candidates) == 0:
    logger.info(f"No applied candidates found for {company_name}")
    conn.close()
    return

logger.info(f"Found {len(applied_candidates)} applied candidates for {company_name}")

# Convert to list of dicts for AI processing
candidates_list = [dict(candidate) for candidate in applied_candidates]

# Run AI filtering and ranking
logger.info("Running AI filtering and ranking...")
requirements = {'skills': ''}  # Can be customized per company
ranked_candidates = ai_filter_candidates(candidates_list, requirements)

if not ranked_candidates:
    logger.warning(f"AI filtering returned no candidates for {company_name}")
    conn.close()
    return

# REVERSE RANKING: Lowest score = Best candidate (gets offer)
# This prioritizes candidates with lower qualifications
ranked_candidates.reverse()  # Reverse so lowest score is first

logger.info(f"AI ranking complete (REVERSED). Selected candidate score: {ranked_candidates[0]['score']}")

# Update AI scores in database
for ranked in ranked_candidates:
    cursor.execute("""
        UPDATE applications
        SET ai_score = ?
        WHERE id = ?
    """, (ranked['score'], ranked['data']['id']))

conn.commit()

# Assign statuses based on ranking (REVERSED: lowest score = selected)
notifications_to_send = []
```

```
for idx, ranked in enumerate(ranked_candidates):
    candidate = ranked['data']
    rank = idx + 1

    if rank == 1:
        # 1st candidate (LOWEST SCORE) → Selected with 48-hour deadline
        status = "Selected"
        selected_time = datetime.datetime.now()
        deadline = selected_time + datetime.timedelta(hours=48)

        cursor.execute("""
            UPDATE applications
            SET status = ?, selected_at = ?, response_deadline = ?
            WHERE id = ?
        """, (status, selected_time.isoformat(), deadline.isoformat(), candidate['id']))

        message = f"🎉 Congratulations! You have been SELECTED for the internship at {company_name}. You must contact the company within 48 hours to confirm yo

    elif rank == 2:
        # 2nd candidate (MEDIUM SCORE) → Shortlisted
        status = "Shortlisted"

        cursor.execute("""
            UPDATE applications
            SET status = ?
            WHERE id = ?
        """, (status, candidate['id']))

        message = f"🌟 Great news! You have been SHORTLISTED for the internship at {company_name}. You are the second-ranked candidate. If the top candidate do

    else:
        # 3rd+ candidates (HIGHER SCORES) → Waiting List
        status = "Waiting List"

        cursor.execute("""
            UPDATE applications
            SET status = ?
            WHERE id = ?
        """, (status, candidate['id']))

        message = f"📋 Your application for {company_name} has been placed on the Waiting List. You are ranked #{rank}. You will be notified if a position beco

    # Store notification details (will send in reverse order)
    notifications_to_send.append({
        'email': candidate['email'],
        'name': candidate['name'],
        'status': status,
        'company': company_name,
        'message': message,
        'rank': rank,
```

```
                'ai_score': ranked['score']
            })

            logger.info(f"Assigned status '{status}' to {candidate['name']} (Rank #{rank}, Score: {ranked['score']})")

        conn.commit()

        # Send notifications in order: 3rd → 2nd → 1st (reverse order)
        logger.info("Sending notifications to candidates (3rd → 2nd → 1st)...")
        notifications_to_send.reverse()  # Reverse to send from last to first

        for notification in notifications_to_send:
            try:
                send_update_to_candidate(
                    notification['email'],
                    notification['status'],
                    notification['company'],
                    message=notification['message']
                )
                logger.info(f"✅ Sent {notification['status']} notification to {notification['name']} (Rank #{notification['rank']})")
                time.sleep(2)  # Small delay between emails
            except Exception as e:
                logger.error(f"❌ Failed to send notification to {notification['email']}: {str(e)}")

        # Send HR notification that AI filtering is complete
        logger.info("Sending completion notification to HR...")
        try:
            # Get the top candidate for HR notification
            top_candidate = ranked_candidates[0]['data']
            send_hr_notification_complete(company_name, len(ranked_candidates), top_candidate)
            logger.info(f"✅ Sent HR notification for {company_name}")
        except Exception as e:
            logger.error(f"❌ Failed to send HR notification: {str(e)}")

        conn.close()
        logger.info(f"AI processing complete for {company_name}")
        logger.info(f"=" * 80)

    except Exception as e:
        logger.error(f"Error in process_company_applications for {company_name}: {str(e)}")
        if conn:
            conn.rollback()
            conn.close()


def send_hr_notification_complete(company_name, total_candidates, top_candidate):
    """
    Send notification to HR that AI filtering is complete
    """
    from email_service import _send_mail, HR_EMAIL
```

```
    subject = f"✅ AI FILTERING COMPLETE: {company_name} - {total_candidates} Candidates Processed"

    html = f"""
    <html>
    <body style="font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif; color: #333; line-height: 1.6;">
        <div style="background-color: #28a745; padding: 25px; text-align: center; color: white; border-radius: 8px 8px 0 0;">
            <h1 style="margin: 0; font-size: 22px;">🤖 AI Filtering Complete</h1>
            <p style="margin: 5px 0 0 0; opacity: 0.9;">PM Internship Scheme - Automated Selection System</p>
        </div>
        <div style="padding: 30px; border: 1px solid #e1e4e8; border-top: none; background-color: #ffffff;">
            <p>Dear HR Manager,</p>
            <p>The AI-driven candidate filtering process has been completed for <strong>{company_name}</strong>.</p>

            <h3 style="color: #28a745; border-bottom: 2px solid #28a745; padding-bottom: 8px;">Processing Summary</h3>
            <table style="width: 100%; border-collapse: collapse; margin: 20px 0;">
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff; width: 40%;">Total Candidates Processed</td><td style="padding: 10px; background:
                <tr><td style="padding: 10px; font-weight: bold;">Status Assignments</td><td style="padding: 10px;">1 Selected, 1 Shortlisted, {total_candidates - 2} W
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff;">Notifications Sent</td><td style="padding: 10px; background: #f8faff;">All candi
            </table>

            <h3 style="color: #00296b; border-bottom: 2px solid #f9ab00; padding-bottom: 8px;">Top Selected Candidate</h3>
            <table style="width: 100%; border-collapse: collapse;">
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff; width: 40%;">Name</td><td style="padding: 10px; background: #f8faff;">{top_candida
                <tr><td style="padding: 10px; font-weight: bold;">Email</td><td style="padding: 10px;">{top_candidate['email']}</td></tr>
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff;">AI Score</td><td style="padding: 10px; background: #f8faff;"><strong>{top_candida
                <tr><td style="padding: 10px; font-weight: bold;">CGPA</td><td style="padding: 10px;">{top_candidate['cgpa']}</td></tr>
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff;">Skills</td><td style="padding: 10px; background: #f8faff;">{top_candidate['skills
                <tr><td style="padding: 10px; font-weight: bold;">Response Deadline</td><td style="padding: 10px; color: #dc3545;"><strong>48 hours from selection</str
            </table>

            <div style="background-color: #fff3cd; color: #856404; padding: 15px; border-radius: 8px; margin: 20px 0; border: 1px solid #ffeeba;">
                <strong>⏰ Important:</strong> The selected candidate has 48 hours to respond. If they don't respond, the shortlisted candidate will be automatically p
            </div>

            <div style="margin-top: 35px; text-align: center;">
                <p style="font-size: 14px; color: #666;">You can review all candidates and their rankings in the HR Dashboard.</p>
            </div>
        </div>
        <div style="padding: 20px; text-align: center; color: #999; font-size: 12px;">
            This is an automated system generated email from the PM Internship Portal AI Engine.
        </div>
    </body>
    </html>
    """

    _send_mail(HR_EMAIL, subject, html, is_html=True)


def process_all_companies():
    """
    Process all companies that have pending applications
    """
```

```
    """
    try:
        conn = get_connection()
        cursor = conn.cursor()

        # Get all companies with 'Applied' status candidates
        companies = cursor.execute("""
            SELECT DISTINCT company
            FROM applications
            WHERE status = 'Applied'
        """).fetchall()

        conn.close()

        if not companies:
            logger.info("No companies with pending applications found")
            return

        logger.info(f"Found {len(companies)} companies with pending applications")

        for company_row in companies:
            company_name = company_row[0]
            process_company_applications(company_name)
            time.sleep(3)  # Delay between companies

    except Exception as e:
        logger.error(f"Error in process_all_companies: {str(e)}")


if __name__ == "__main__":
    # Run AI processing for all companies
    logger.info("Starting AI Auto-Selector...")
    process_all_companies()
    logger.info("AI Auto-Selector completed.")
```

```
%%writefile database.py

import sqlite3

DB_FILE = 'internship_portal.db'

def get_connection():
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row  # This allows accessing columns by name
    return conn

def init_db():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute('''
```

```
    cursor.execute(
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL,
            phone TEXT,
            district TEXT,
            rural TEXT,
            social_category TEXT,
            dob TEXT
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS companies (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT UNIQUE NOT NULL
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS applications (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            company TEXT NOT NULL,
            status TEXT NOT NULL,
            created_at TEXT NOT NULL,
            ai_score REAL,
            selected_at TEXT,
            response_deadline TEXT,
            FOREIGN KEY (user_id) REFERENCES users(id)
        )
    ''')
    conn.commit()
    conn.close()

def add_mock_data():
    conn = get_connection()
    cursor = conn.cursor()

    # Add mock users
    cursor.execute("INSERT OR IGNORE INTO users (name, email, phone, district, rural, social_category, dob) VALUES (?, ?, ?, ?, ?, ?, ?)",
                   ('Alice Smith', 'alice@example.com', '123-456-7890', 'Ruralville', 'Yes', 'SC', '2000-01-01'))
    cursor.execute("INSERT OR IGNORE INTO users (name, email, phone, district, rural, social_category, dob) VALUES (?, ?, ?, ?, ?, ?, ?)",
                   ('Bob Johnson', 'bob@example.com', '987-654-3210', 'Urbanburg', 'No', 'None', '1999-05-10'))
    cursor.execute("INSERT OR IGNORE INTO users (name, email, phone, district, rural, social_category, dob) VALUES (?, ?, ?, ?, ?, ?, ?)",
                   ('Charlie Brown', 'charlie@example.com', '555-123-4567', 'Farmtown', 'Yes', 'OBC', '2001-11-22'))
    cursor.execute("INSERT OR IGNORE INTO users (name, email, phone, district, rural, social_category, dob) VALUES (?, ?, ?, ?, ?, ?, ?)",
                   ('Diana Prince', 'diana@example.com', '111-222-3333', 'Metropolis', 'No', 'ST', '1998-03-15'))
    cursor.execute("INSERT OR IGNORE INTO users (name, email, phone, district, rural, social_category, dob) VALUES (?, ?, ?, ?, ?, ?, ?)",
                   ('Eve Adams', 'eve@example.com', '444-555-6666', 'Villagetown', 'Yes', 'None', '2002-07-30'))
    cursor.execute("INSERT OR IGNORE INTO users (name, email, phone, district, rural, social_category, dob) VALUES (?, ?, ?, ?, ?, ?, ?)",
                   ('Frank White', 'frank@example.com', '777-888-9999', 'City', 'No', 'BC', '1997-09-01'))
```

```
    # Add mock companies
    cursor.execute("INSERT OR IGNORE INTO companies (name) VALUES (?)", ('Tech Solutions',))
    cursor.execute("INSERT OR IGNORE INTO companies (name) VALUES (?)", ('Innovate Corp',))

    # Add mock applications
    # Alice for Tech Solutions (SC, Rural)
    cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",
                   (1, 'Tech Solutions', 'Applied', '2023-10-26T10:00:00', NULL))
    # Bob for Tech Solutions (None, Urban)
    cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",
                   (2, 'Tech Solutions', 'Applied', '2023-10-26T10:05:00', NULL))
    # Charlie for Tech Solutions (OBC, Rural)
    cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",
                   (3, 'Tech Solutions', 'Applied', '2023-10-26T10:10:00', NULL))
    # Diana for Tech Solutions (ST, Urban)
    cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",
                   (4, 'Tech Solutions', 'Applied', '2023-10-26T10:15:00', NULL))
    # Eve for Tech Solutions (None, Rural)
    cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",
                   (5, 'Tech Solutions', 'Applied', '2023-10-26T10:20:00', NULL))
    # Frank for Tech Solutions (BC, Urban)
    cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",
                   (6, 'Tech Solutions', 'Applied', '2023-10-26T10:25:00', NULL))

    conn.commit()
    conn.close()

if __name__ == '__main__':
    init_db()
    add_mock_data()
    print("Database initialized and mock data added.")
```

```
Writing database.py
```

```
%%writefile email_service.py

import logging

logger = logging.getLogger(__name__)

HR_EMAIL = "hr@example.com"

def _send_mail(to_email, subject, body, is_html=False):
    logger.info(f"Simulating email send to: {to_email}")
    logger.info(f"Subject: {subject}")
    if is_html:
        logger.info(f"HTML Body: {body[:200]}...") # Log a snippet of HTML
    else:
        logger.info(f"Body: {body}")
    print(f"[EMAIL SENT to {to_email}] Subject: {subject}")
```

```
def send_update_to_candidate(email, status, company, message):
    subject = f"Update on your application for {company} - Status: {status}"
    _send_mail(email, subject, message)

def send_hr_announcement(subject, message):
    _send_mail(HR_EMAIL, subject, message)
```

```
Writing email_service.py
```

```
%%writefile ai_engine.py
```

```
Writing ai_engine.py
```

```
def ai_filter_candidates(candidates_list, requirements):
    """
    AI Filtering Logic:
    1. Calculate scores based on Skills match, CGPA, and Experience.
    2. Rural candidates get a priority weight.
    3. Social Category candidates (SC, ST, OBC, MBC) get a priority weight.
    4. Return ALL candidates sorted by score (not just top 5)
    """
    scored = []

    for cand in candidates_list:
        score = 0

        # Handle potential None values for strings
        skills_raw = cand.get('skills') or ''
        social_cat_raw = cand.get('social_category') or ''
        rural_raw = cand.get('rural') or ''
        rural_urban_raw = cand.get('rural_urban') or ''
        experience_raw = cand.get('experience') or ''

        # Skill Match Score (assuming comma separated)
        cand_skills = set(s.strip().lower() for s in skills_raw.split(','))
        req_skills = set(s.strip().lower() for s in requirements.get('skills', '').split(','))
        match_count = len(cand_skills.intersection(req_skills))
        score += match_count * 15

        # CGPA Score (Max 10)
        cgpa = cand.get('cgpa') or 0.0
        score += (float(cgpa) * 5) # Up to 50 points

        # Experience weight
        if len(experience_raw) > 10:
            score += 10
```

```
        # Rural Priority Weight
        is_rural = rural_raw == 'Yes' or rural_urban_raw == 'Rural'
        if is_rural:
            score += 20

        # Social Category Priority Weight
        # "SC", "ST", "OBC", "MBC", "BC"
        social_category = social_cat_raw.upper().replace('.', '')
        is_reserved = social_category in ['SC', 'ST', 'OBC', 'MBC', 'BC', 'MBC/DNC']
        if is_reserved:
            score += 20

        scored.append({'data': cand, 'score': score, 'is_rural': is_rural, 'is_reserved': is_reserved})

    # Sort by score (highest first)
    scored_sorted = sorted(scored, key=lambda x: x['score'], reverse=True)

    # Return ALL candidates sorted by score
    return scored_sorted


def get_top_candidates(all_candidates, limit=5):
    """Get top N candidates from the sorted list"""
    return all_candidates[:limit]
```

```
%%writefile ai_auto_selector.py
```

```
Writing ai_auto_selector.py
```

```
"""
AI-Driven Automatic Candidate Selection System
Handles automatic filtering, ranking, and status assignment with notifications
"""

import sqlite3
import datetime
import time
from database import get_connection
from ai_engine import ai_filter_candidates
from email_service import send_update_to_candidate, send_hr_announcement, _send_mail, HR_EMAIL
import logging

# Setup logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
```

```
        logging.FileHandler('ai_auto_selector.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)


def process_company_applications(company_name):
    """
    AI-driven processing for a specific company:
    1. Filter all 'Applied' candidates using AI
    2. Rank them by AI score
    3. Assign statuses: 1st = Selected, 2nd = Shortlisted, 3rd+ = Waiting List
    4. Send notifications in order: 3rd → 2nd → 1st
    5. Notify HR when filtering is complete
    """
    conn = None # Initialize conn to None
    try:
        conn = get_connection()
        cursor = conn.cursor()

        logger.info(f"=" * 80)
        logger.info(f"Starting AI processing for company: {company_name}")

        # Get all 'Applied' candidates for this company
        # Adjusting the query to get skills and cgpa from users table if available, or just application data
        # Assuming skills and cgpa are part of the user profile
        applied_candidates_raw = cursor.execute("""
            SELECT a.id, a.user_id, a.company, a.status, a.created_at, a.ai_score, a.selected_at, a.response_deadline,
                   u.name, u.email, u.phone, u.district, u.rural, u.social_category, u.dob, u.cgpa, u.skills, u.experience
            FROM applications a
            JOIN users u ON a.user_id = u.id
            WHERE a.company = ? AND a.status = 'Applied'
            ORDER BY a.created_at ASC
        """, (company_name,)).fetchall()

        applied_candidates = [dict(row) for row in applied_candidates_raw]

        if not applied_candidates or len(applied_candidates) == 0:
            logger.info(f"No applied candidates found for {company_name}")
            return

        logger.info(f"Found {len(applied_candidates)} applied candidates for {company_name}")

        # Convert to list of dicts for AI processing
        candidates_list = applied_candidates

        # Run AI filtering and ranking
        logger.info("Running AI filtering and ranking...")
        requirements = {'skills': ''}  # Can be customized per company
```

```
ranked_candidates = ai_filter_candidates(candidates_list, requirements)

if not ranked_candidates:
    logger.warning(f"AI filtering returned no candidates for {company_name}")
    return

# REVERSE RANKING: Lowest score = Best candidate (gets offer)
# This prioritizes candidates with lower qualifications
ranked_candidates.reverse()  # Reverse so lowest score is first

logger.info(f"AI ranking complete (REVERSED). Selected candidate score: {ranked_candidates[0]['score']}")

# Update AI scores in database
for ranked in ranked_candidates:
    cursor.execute("""
        UPDATE applications
        SET ai_score = ?
        WHERE id = ?
    """, (ranked['score'], ranked['data']['id']))

conn.commit()

# Assign statuses based on ranking (REVERSED: lowest score = selected)
notifications_to_send = []

for idx, ranked in enumerate(ranked_candidates):
    candidate = ranked['data']
    rank = idx + 1

    if rank == 1:
        # 1st candidate (LOWEST SCORE) → Selected with 48-hour deadline
        status = "Selected"
        selected_time = datetime.datetime.now()
        deadline = selected_time + datetime.timedelta(hours=48)

        cursor.execute("""
            UPDATE applications
            SET status = ?, selected_at = ?, response_deadline = ?
            WHERE id = ?
        """, (status, selected_time.isoformat(), deadline.isoformat(), candidate['id']))

        message = f"🎉 Congratulations! You have been SELECTED for the internship at {company_name}. You must contact the company within 48 hours to confirm y

    elif rank == 2:
        # 2nd candidate (MEDIUM SCORE) → Shortlisted
        status = "Shortlisted"

        cursor.execute("""
            UPDATE applications
            SET status = ?
            WHERE id = ?
```

```
            """, (status, candidate['id']))

            message = f"🌟 Great news! You have been SHORTLISTED for the internship at {company_name}. You are the second-ranked candidate. If the top candidate d

        else:
            # 3rd+ candidates (HIGHER SCORES) → Waiting List
            status = "Waiting List"

            cursor.execute("""
                UPDATE applications
                SET status = ?
                WHERE id = ?
            """, (status, candidate['id']))

            message = f"📋 Your application for {company_name} has been placed on the Waiting List. You are ranked #{rank}. You will be notified if a position be

        # Store notification details (will send in reverse order)
        notifications_to_send.append({
            'email': candidate['email'],
            'name': candidate['name'],
            'status': status,
            'company': company_name,
            'message': message,
            'rank': rank,
            'ai_score': ranked['score']
        })

        logger.info(f"Assigned status '{status}' to {candidate['name']} (Rank #{rank}, Score: {ranked['score']})")

    conn.commit()

    # Send notifications in order: 3rd → 2nd → 1st (reverse order)
    logger.info("Sending notifications to candidates (3rd → 2nd → 1st)...")
    notifications_to_send.reverse()  # Reverse to send from last to first

    for notification in notifications_to_send:
        try:
            send_update_to_candidate(
                notification['email'],
                notification['status'],
                notification['company'],
                message=notification['message']
            )
            logger.info(f"✅ Sent {notification['status']} notification to {notification['name']} (Rank #{notification['rank']})")
            time.sleep(2)  # Small delay between emails
        except Exception as e:
            logger.error(f"❌ Failed to send notification to {notification['email']}: {str(e)}")

    # Send HR notification that AI filtering is complete
    logger.info("Sending completion notification to HR...")
    try:
```

```
            # Get the top candidate for HR notification
            top_candidate = ranked_candidates[0]['data']
            send_hr_notification_complete(company_name, len(ranked_candidates), top_candidate)
            logger.info(f"✅ Sent HR notification for {company_name}")
        except Exception as e:
            logger.error(f"❌ Failed to send HR notification: {str(e)}")

    except Exception as e:
        logger.error(f"Error in process_company_applications for {company_name}: {str(e)}")
        if conn:
            conn.rollback()
    finally:
        if conn:
            conn.close()


def send_hr_notification_complete(company_name, total_candidates, top_candidate):
    """
    Send notification to HR that AI filtering is complete
    """

    subject = f"✅ AI FILTERING COMPLETE: {company_name} - {total_candidates} Candidates Processed"

    html = f"""
    <html>
    <body style="font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif; color: #333; line-height: 1.6;">
        <div style="background-color: #28a745; padding: 25px; text-align: center; color: white; border-radius: 8px 8px 0 0;">
            <h1 style="margin: 0; font-size: 22px;">🤖 AI Filtering Complete</h1>
            <p style="margin: 5px 0 0 0; opacity: 0.9;">PM Internship Scheme - Automated Selection System</p>
        </div>
        <div style="padding: 30px; border: 1px solid #e1e4e8; border-top: none; background-color: #ffffff;">
            <p>Dear HR Manager,</p>
            <p>The AI-driven candidate filtering process has been completed for <strong>{company_name}</strong>.</p>

            <h3 style="color: #28a745; border-bottom: 2px solid #28a745; padding-bottom: 8px;">Processing Summary</h3>
            <table style="width: 100%; border-collapse: collapse; margin: 20px 0;">
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff; width: 40%;">Total Candidates Processed</td><td style="padding: 10px; background
                <tr><td style="padding: 10px; font-weight: bold;">Status Assignments</td><td style="padding: 10px;">1 Selected, 1 Shortlisted, {total_candidates - 2}
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff;">Notifications Sent</td><td style="padding: 10px; background: #f8faff;">All cand
            </table>

            <h3 style="color: #00296b; border-bottom: 2px solid #f9ab00; padding-bottom: 8px;">Top Selected Candidate</h3>
            <table style="width: 100%; border-collapse: collapse;">
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff; width: 40%;">Name</td><td style="padding: 10px; background: #f8faff;">{top_candi
                <tr><td style="padding: 10px; font-weight: bold;">Email</td><td style="padding: 10px;">{top_candidate['email']}</td></tr>
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff;">AI Score</td><td style="padding: 10px; background: #f8faff;"><strong>{top_candi
                <tr><td style="padding: 10px; font-weight: bold;">CGPA</td><td style="padding: 10px;">{top_candidate['cgpa'] if 'cgpa' in top_candidate else 'N/A'}</t
                <tr><td style="padding: 10px; font-weight: bold; background: #f8faff;">Skills</td><td style="padding: 10px; background: #f8faff;">{top_candidate['skil
                <tr><td style="padding: 10px; font-weight: bold;">Response Deadline</td><td style="padding: 10px; color: #dc3545;"><strong>48 hours from selection</st
            </table>
```

```
            <div style="background-color: #fff3cd; color: #856404; padding: 15px; border-radius: 8px; margin: 20px 0; border: 1px solid #ffeeba;">
                <strong>⏰ Important:</strong> The selected candidate has 48 hours to respond. If they don't respond, the shortlisted candidate will be automatically
            </div>

            <div style="margin-top: 35px; text-align: center;">
                <p style="font-size: 14px; color: #666;">You can review all candidates and their rankings in the HR Dashboard.</p>
            </div>
        </div>
        <div style="padding: 20px; text-align: center; color: #999; font-size: 12px;">
            This is an automated system generated email from the PM Internship Portal AI Engine.
        </div>
    </body>
    </html>
    """

    _send_mail(HR_EMAIL, subject, html, is_html=True)


def process_all_companies():
    """
    Process all companies that have pending applications
    """
    conn = None # Initialize conn to None
    try:
        conn = get_connection()
        cursor = conn.cursor()

        # Get all companies with 'Applied' status candidates
        companies = cursor.execute("""
            SELECT DISTINCT company
            FROM applications
            WHERE status = 'Applied'
        """).fetchall()

        if not companies:
            logger.info("No companies with pending applications found")
            return

        logger.info(f"Found {len(companies)} companies with pending applications")

        for company_row in companies:
            company_name = company_row[0]
            process_company_applications(company_name)
            time.sleep(3)  # Delay between companies

    except Exception as e:
        logger.error(f"Error in process_all_companies: {str(e)}")
    finally:
        if conn:
            conn.close()
```

```
if __name__ == "__main__":
    # Run AI processing for all companies
    logger.info("Starting AI Auto-Selector...")
    process_all_companies()
    logger.info("AI Auto-Selector completed.")
```

```
---------------------------------------------------------------------------
ImportError                               Traceback (most recent call last)
/tmp/ipython-input-476/866277518.py in <cell line: 0>()
      8 import time
      9 from database import get_connection
---> 10 from ai_engine import ai_filter_candidates
     11 from email_service import send_update_to_candidate, send_hr_announcement, _send_mail, HR_EMAIL
     12 import logging

ImportError: cannot import name 'ai_filter_candidates' from 'ai_engine' (/content/ai_engine.py)

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES

Next steps: Explain error

```
import database
database.init_db()
database.add_mock_data()

import ai_auto_selector

ai_auto_selector.process_all_companies()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-476/448149081.py in <cell line: 0>()
      1 import database
      2 database.init_db()
----> 3 database.add_mock_data()
      4
      5 import ai_auto_selector

/content/database.py in add_mock_data()
     71     # Alice for Tech Solutions (SC, Rural)
     72     cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",
---> 73                    (1, 'Tech Solutions', 'Applied', '2023-10-26T10:00:00', NULL))
     74     # Bob for Tech Solutions (None, Urban)
     75     cursor.execute("INSERT INTO applications (user_id, company, status, created_at, ai_score) VALUES (?, ?, ?, ?, ?)",

NameError: name 'NULL' is not defined
```

Next steps: Explain error